

How the Desert Runner Works

A picture book explanation of the OpenGL code

August 18, 2026

CONTENTS

The Big Secret: It Is a Flipbook	
The Drawing Board	
Only four shapes exist	
Painting Rules: The Last One Wins	
The Treadmill: Why the Desert Slides Past	
The conveyor belt	
Near things rush, far things crawl	
The Paper Puppet: How the Girl Is Built	
Three magic words	
One whole leg, in code	
The parts she is made of	
The Swing: How One Number Makes Her Run	
The Jump: Real Gravity, Not Pretend	
Changing shape in mid air	
The Scoreboard	
Two Files, One Game	
The header is a menu	
The word static means private	
Every Function, One Line Each	
desert_scene.cpp	
player.cpp	

Build It and Run It

Try It Yourself

The Big Secret: It Is a Flipbook

If you draw a stick figure on the corner of every page in a notebook, moving him a tiny bit each time, then flip the pages fast, he walks. That is a flipbook. Nothing on the page ever moves. You just see new pictures very quickly.

This whole program is a flipbook. It draws a brand new desert picture **60 times every second**. Nothing in it slides or walks. Each picture is drawn from scratch, with everything in a slightly different spot.

Two functions do this job. `display()` is the artist: it paints one complete picture. `tick()` is the page turner.

Here is the page turner, exactly as it appears in the code.

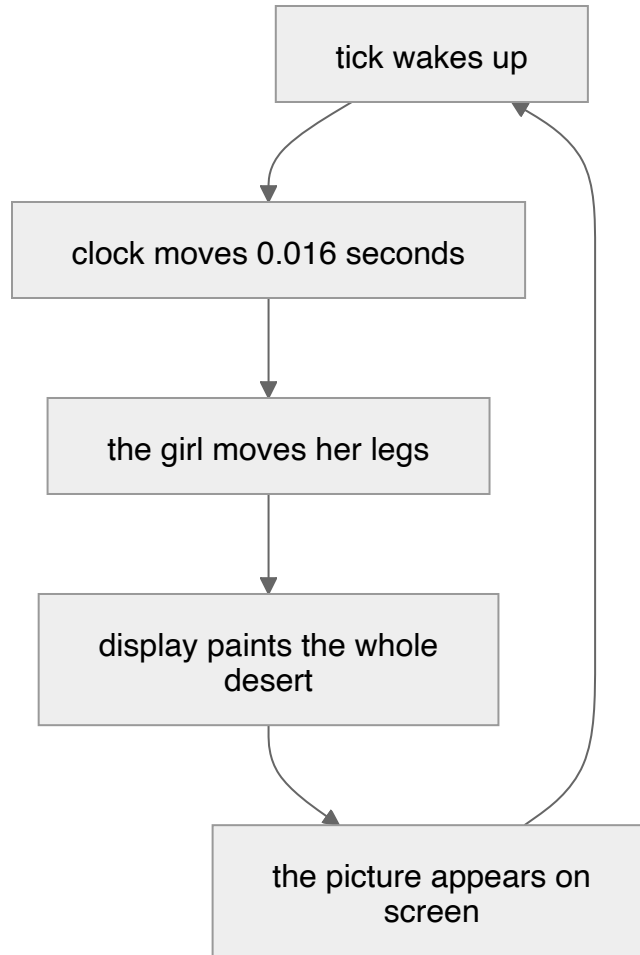
```
void tick(int)
{
    if (!gPaused)
    {
        gTime    += 0.016f;
        gElapsed += 0.016f;
        playerUpdate(0.016f);
    }
    glutPostRedisplay();
    glutTimerFunc(16, tick, 0);
}
```

Read it line by line.

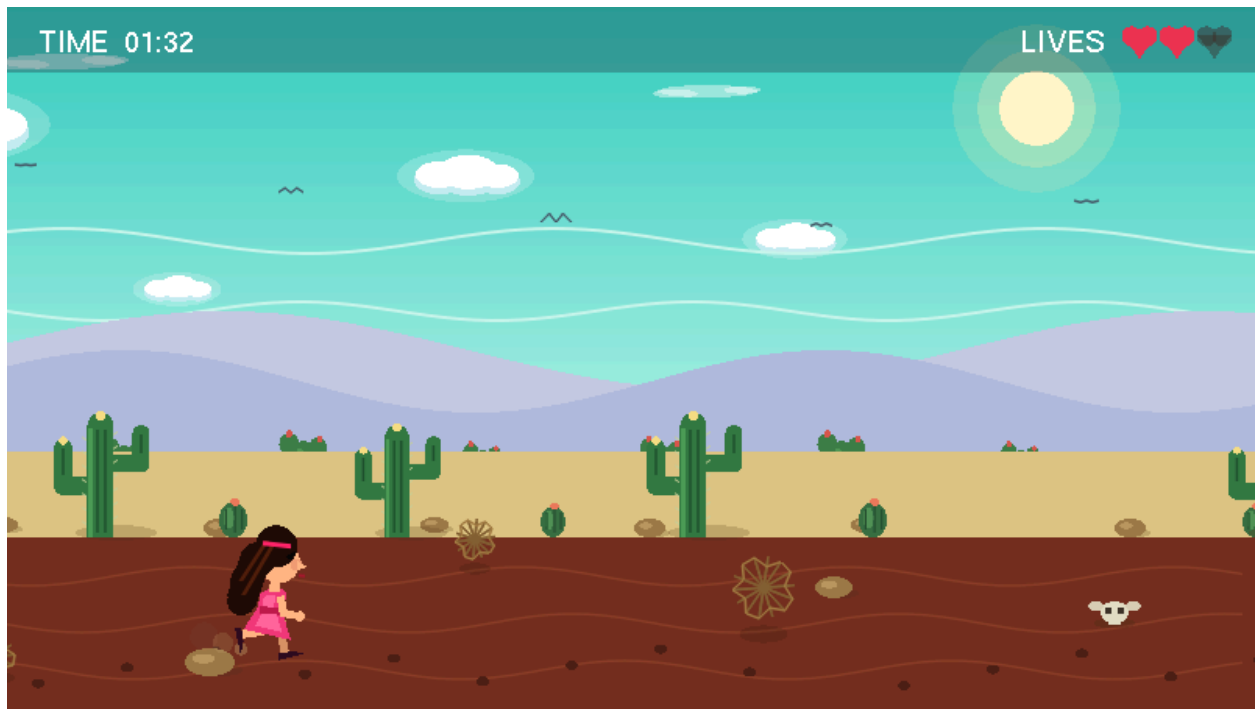
Line	In plain words
<code>if (!gPaused)</code>	if the game is not paused, let time move
<code>gTime += 0.016f</code>	add one sixtieth of a second to the world clock
<code>playerUpdate(0.016f)</code>	tell the girl that much time has passed, so she moves her legs
<code>glutPostRedisplay()</code>	ask for a fresh picture, which runs <code>display()</code>
<code>glutTimerFunc(16, tick, 0)</code>	come back and do all of this again in 16 milliseconds

That last line is the clever bit. `tick` asks for `tick` to be called again. It sets up its own future, over and over, and that is what keeps the game alive.

How the Desert Runner Works



The flipbook loop, 60 times every second



The Drawing Board

Imagine a big sheet of graph paper, **800 squares wide and 450 squares tall**. The bottom left corner is `(0, 0)`. The top right corner is `(800, 450)`.

`init()` sets up that paper. It runs once, at the very start.

```
void init()
{
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glEnable(GL_LINE_SMOOTH);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0, 800, 0, 450);
    glMatrixMode(GL_MODELVIEW);
}
```

The important line is `gluOrtho2D(0, 800, 0, 450)`. It means “this is my paper, this is how big it is”. Every position in the whole game is measured on it. The girl stands at 170 across. The scoreboard sits at 408 up, near the ceiling.

Only four shapes exist

Here is the surprising part. A desert, a cactus, a sun, a girl, and a heart are all built from **four shapes** and nothing else.

Shape	Function	What it really is
Box	<code>drawRect</code>	four corners joined up
Oval	<code>drawEllipse</code>	a circle that got squashed
Circle	<code>drawCircle</code>	an oval that did not get squashed
Sausage	<code>drawCapsule</code>	a box with a round hat on top

Now look at the oval, because it hides a secret.


```
void drawEllipse(float cx, float cy, float rx, float ry)
{
    glBegin(GL_POLYGON);
    for (int i = 0; i < 28; i++)
    {
        float angle = (i * 2 * PI) / 28;
        glVertex2f(cx + rx * cos(angle), cy + ry * sin(angle));
    }
    glEnd();
}
```

A computer cannot actually draw a curve. So this walks around in a ring, stopping **28 times**, and joins those 28 dots with straight lines. `cos` gives the across position, `sin` gives the up position, and together they walk in a perfect circle. Twenty eight straight lines in a ring look exactly like a circle to your eye.

Every round thing in this game, the sun, a cloud, a wheel of a tumbleweed, a heart, her head, is secretly a twenty eight sided shape.

Painting Rules: The Last One Wins

Painting on a wall has one rule. Whatever you paint last covers up what is underneath. Computers draw the same way, and this matters more than anything else in the program.

If the girl is drawn **before** the ground, the ground paints right over her and she vanishes. If the scoreboard is drawn **before** the sky, the sky swallows it.

So the order inside `display()` is not random. It runs from the furthest away thing to the closest thing, exactly like painting a stage set.

```
void display()
{
    glClear(GL_COLOR_BUFFER_BIT);    // wipe the page clean

    drawSky();                      // 1. furthest away
    drawSun(660, 385);
    drawWispCloud(...); drawCloud(...);
    drawBird(...);
    drawHill(...);                  // purple hills
    drawSmallCactus(...);           // little cacti behind

    drawGround();                   // 2. the sand and dirt

    drawRock(...); drawBarrelCactus(...); drawCactus(...);

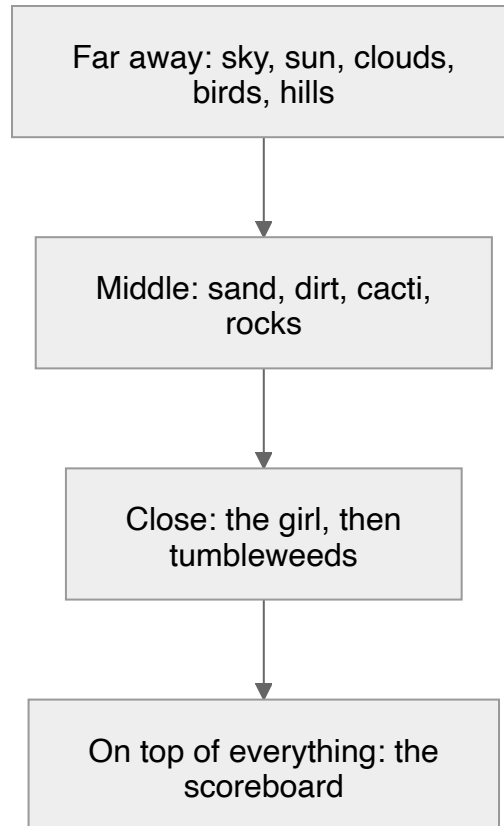
    drawSkull(...); drawRock(...); // 3. foreground junk

    glPushMatrix();                 // 4. the girl
        glTranslatef(170, 30, 0);
        glScalef(0.75f, 0.75f, 1.0f);
        drawPlayer(0, 0);
    glPopMatrix();

    drawTumbleweed(...);            // 5. rolls past in front of her

    drawHUD();                       // 6. always last, always on top

    glutSwapBuffers();              // show the finished picture
}
```



Furthest away is painted first, closest is painted last

`glutSwapBuffers()` on the last line is worth knowing about. The picture is painted on a hidden page, not the one you are looking at. Only when it is completely finished does the program swap the two pages over. That is why you never catch it half drawn, and why the animation looks smooth instead of flickery.

The Treadmill: Why the Desert Slides Past

Here is a secret about the girl. **She never moves.** She stays at 170 across, forever, running on the spot like someone on a treadmill.

The desert is what moves. The whole world slides to the left, so it looks like she is racing to the right.

The conveyor belt

One small function does all the sliding.

```
static float wrapX(float x0, float speed, float margin)
{
    float span = 800.0f + 2.0f * margin;
    float x = fmodf(x0 - speed * gTime, span);
    if (x < 0.0f) x += span;
    return x - margin;
}
```

Think of a luggage belt at an airport. A bag rolls to the left, falls off the end, and gets carried back round to the right to start again.

Line	In plain words
<code>span = 800 + 2 * margin</code>	the belt is a bit wider than the screen, so things vanish before they wrap
<code>speed * gTime</code>	how far this thing has travelled since the game began
<code>fmodf(...)</code>	the wrap around. "What is left over after going round the loop?"
<code>if (x < 0) x += span</code>	keeps the answer positive

Six cacti can therefore look like a hundred cacti. They are the same six, coming round again and again. Here is how a cactus is asked to slide:

```
drawCactus(wrapX(120 + i * 190.0f, 60.0f, 70.0f), 110, 0.9f);
//          start here    spacing  speed  margin
```

Near things rush, far things crawl

Look out of a car window. Fence posts by the road whizz past so fast you cannot count them. A mountain far away barely moves at all. Your brain reads that difference as distance.

The program copies this exactly. Everything moves left, but at different speeds.

What	Speed	Why
Purple hills	6 and 12	very far away
Little cacti behind	20	far
Big cacti and rocks	60	middle
Dirt and pebbles	95	close, under her feet
Tumbleweeds	120 to 185	closest, and rolling

Compare these two pictures. Same desert, about one second apart. The tumbleweed has travelled a long way. The purple hills have hardly shifted.



The Paper Puppet: How the Girl Is Built

Cut a body, two arms and two legs out of paper. Join them with split pins at the shoulders and hips. Now you can swing an arm without cutting a new one. That is exactly how the girl works.

Three magic words

Command	In plain words
<code>glPushMatrix()</code>	remember exactly where I am standing
<code>glRotatef(30, 0, 0, 1)</code>	turn 30 degrees
<code>glPopMatrix()</code>	go back to where I was standing

Because you can always go back, one leg turning does not knock the head sideways.

One whole leg, in code

```
static void drawLeg(float hipX, float hipY, float hipA, float kneeA,
                  float r, float g, float b)
{
    const float thigh = 19.0f, shin = 18.0f, w = 8.0f;

    glPushMatrix();
    glTranslatef(hipX, hipY, 0);
    glRotatef(hipA, 0, 0, 1);

    glColor3f(r, g, b);
    drawRect(-w * 0.5f, -thigh, w * 0.5f, 0);
    drawCircle(0, -thigh, w * 0.5f);           // knee joint keeps the
bend solid

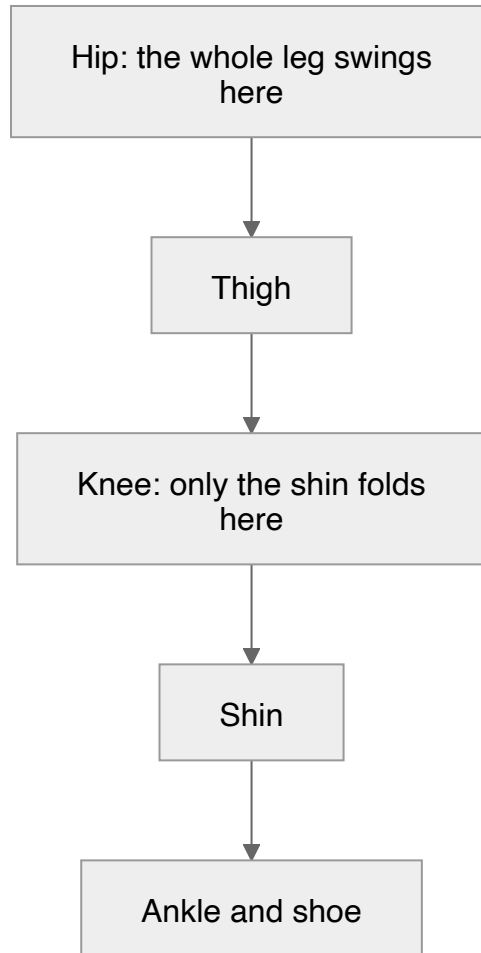
    glTranslatef(0, -thigh, 0);
    glRotatef(kneeA, 0, 0, 1);

    glColor3f(r, g, b);
    drawRect(-w * 0.45f, -shin, w * 0.45f, 0);
    glTranslatef(0, -shin, 0);
    drawShoe(w);

    glPopMatrix();
}
```

Follow what happens, step by step:

1. **Go to the hip.** `glTranslatef(hipX, hipY, 0)` moves the pen to her hip.
2. **Turn the whole leg.** `glRotatef(hipA, ...)` swings everything that follows.
3. **Draw the thigh** as a box hanging downwards, plus a circle at the knee so the joint stays solid when it bends.
4. **Move down to the knee** and turn again by `kneeA`. This second turn is added on top of the first one, which is exactly how a real knee behaves.
5. **Draw the shin**, move down to the ankle, and draw the shoe there.
6. `glPopMatrix()` puts the pen back where it started, ready for the next body part.



One leg is a chain of joints

The parts she is made of

Function	Draws
<code>drawLeg</code>	thigh, knee, shin, and it calls the shoe
<code>drawShoe</code>	the little dark shoe at the ankle
<code>drawArm</code>	upper arm, elbow, forearm, hand
<code>drawBodice</code>	the top of her dress, from waist to shoulders
<code>drawDress</code>	the skirt, which lifts and flaps while she runs
<code>drawDust</code>	three puffs of sand kicked up behind her
<code>drawLandPuff</code>	the bigger puff when she lands from a jump
<code>drawPlayerPose</code>	the boss. Calls all of the above in the right order

The Swing: How One Number Makes Her Run

Watch a child on a swing. Forward, back, forward, back, always smooth, never jerky. Write down how far forward the swing is at every moment and you have drawn a **sine wave**. In code that is `sin()`.

The girl has one number called `p`, short for phase. It counts up forever. Every single thing about her running is worked out from that one number.

```
// ---- the cycle -----
float bob    = 4.0f * fabs(sin(p)) * (1 - air); // planted at mid-
stance, up in mid-flight
float lean   = mixf(5.0f, 8.0f, air);           // she leans into the
run, harder in the air
float hipF   = 32 * sin(p);                     // front leg
float hipB   = 32 * sin(p + PI);                // back leg, half a
cycle behind
float kneeF  = -30 - 30 * cos(p + 0.6f);        // folds after toe-
off, straight when reaching
float kneeB  = -30 - 30 * cos(p + PI + 0.6f);
float armF   = 30 * sin(p + PI);                // arms oppose the
legs
float armB   = 30 * sin(p);
float hem    = 22 + 2 * sin(p);
float flow   = 4 + 3 * sin(p);
float hairA  = -10 + 6 * sin(p);

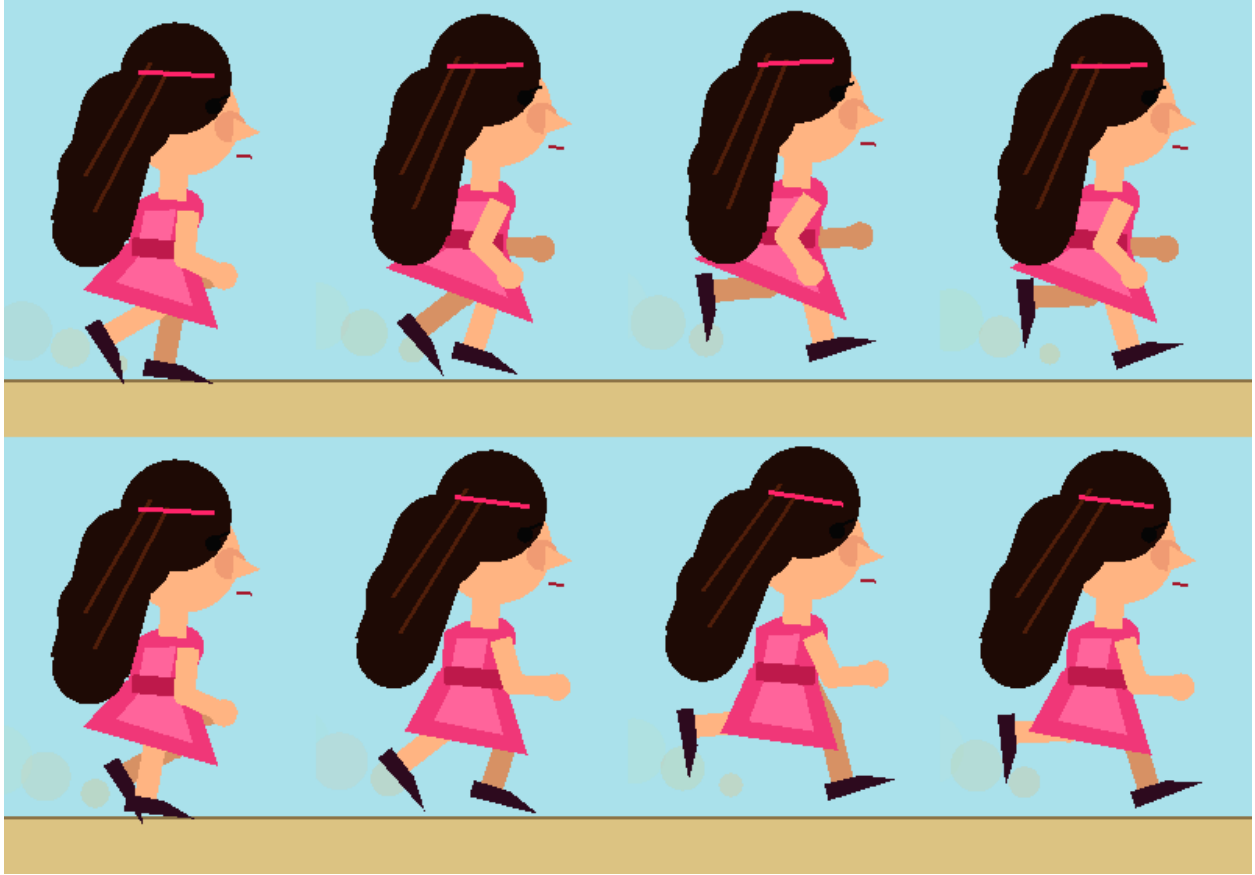
// leap pose: lead leg tucked up in front, trailing leg kicked out
behind,
// lead arm thrown overhead, skirt and hair blown up by the rise
hipF = mixf(hipF, 48.0f, air);
kneeF = mixf(kneeF, -55.0f, air);
hipB = mixf(hipB, -38.0f, air);
kneeB = mixf(kneeB, -78.0f, air);
armF = mixf(armF, 60.0f, air);
armB = mixf(armB, -55.0f, air);
hem = mixf(hem, 28.0f, air);
flow = mixf(flow, 9.0f, air);
hairA = mixf(hairA, -16.0f, air);
```

Line	What you see
<code>hipF = 32 * sin(p)</code>	the front leg swings 32 degrees forward and back
<code>hipB = 32 * sin(p + PI)</code>	<code>PI</code> is half a circle, so the back leg always does the opposite
<code>kneeF = -30 - 30 * cos(...)</code>	always a negative number, so the knee can only fold backwards
<code>armF = 30 * sin(p + PI)</code>	arms swing against the legs, just like yours do
<code>bob = 4 * fabs(sin(p))</code>	<code>fabs</code> throws away the minus sign, so she bounces up twice per stride
<code>hem, flow, hairA</code>	skirt and hair sway with the same wave

Adding `PI` is the whole trick for the second leg. One number, two legs, no extra work. The block below those lines is the jump, which the next chapter explains.

Here is one complete run cycle, sliced into eight pictures. Read the top row left to right, then the bottom row. Picture 1 and the picture after 8 are identical, which is why it can loop forever without a bump.

How the Desert Runner Works



The Jump: Real Gravity, Not Pretend

Throw a ball straight up. It leaves your hand fast, slows down, stops for an instant, then falls back faster and faster. Nobody tells the ball where to be. Gravity just pulls on it every moment.

The jump uses that same idea instead of a fixed animation. Pressing space calls this:

```
void playerJump()
{
    if (gJumpY == 0.0f && gJumpV == 0.0f) gJumpV = JUMP_V;
}
```

It gives her an upward speed of 330, but only if she is standing still on the ground. That `if` is what stops a double jump in mid air.

Then gravity takes over, every single frame:

```
void playerUpdate(float dt)
{
    gPhase += gRate * dt;
    if (gPhase > 2 * PI) gPhase -= 2 * PI;

    if (gJumpY > 0.0f || gJumpV > 0.0f)
    {
        gJumpV -= GRAVITY * dt;
        gJumpY += gJumpV * dt;
        if (gJumpY <= 0.0f) // touchdown
        {
            gJumpY = 0.0f;
            gJumpV = 0.0f;
            gPhase = 0.0f; // land on a planted foot
            gLand = 0.35f;
        }
    }
    if (gLand > 0.0f) gLand -= dt;
}
```

The two lines that matter are these:

```
gJumpV -= GRAVITY * dt;    // speed drops by 900 for every second that
                             passes
gJumpY += gJumpV * dt;     // height changes by whatever the speed is
                             right now
```

That is all real physics is. Nothing draws an arc on purpose. The arc appears by itself: fast up, slow at the top, fast down. She reaches about 60 high and is in the air for about three quarters of a second.

Notice `gPhase = 0.0f` on touchdown. That resets the run cycle to the exact moment a foot is flat on the floor, so she never lands with both feet in mid air.

Changing shape in mid air

A running pose looks silly floating in the sky, so she changes shape. But snapping instantly into a jump pose would look broken. So there is a **mixing dial**.

```
void drawPlayer(float x, float y)
{
    if (gLand > 0.0f) drawLandPuff(x, y, gLand / 0.35f);

    float air = gJumpY / 14.0f;           // pose blends in as she
    leaves the ground
    if (air > 1.0f) air = 1.0f;

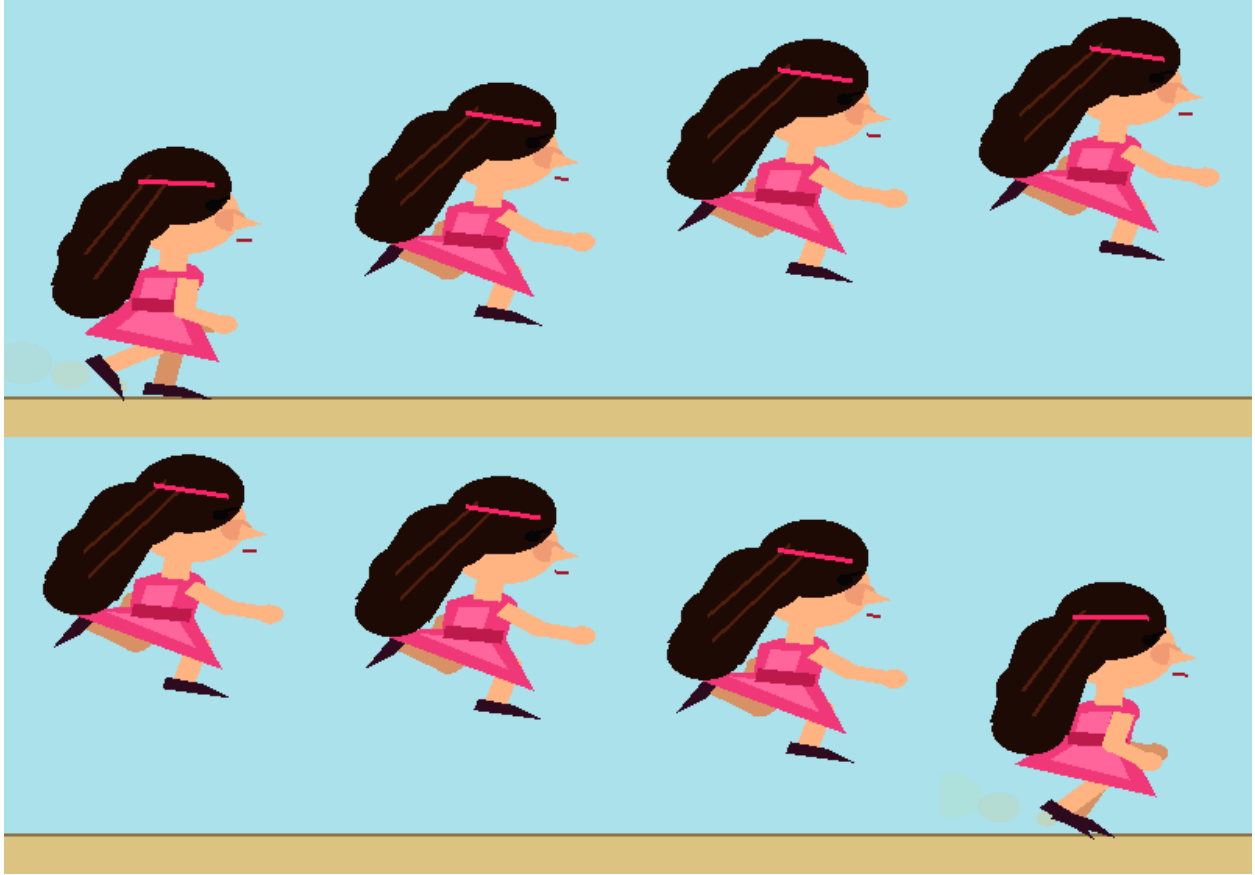
    drawPlayerPose(x, y + gJumpY, gPhase, air);
}
```

`air` is 0 on the ground and 1 once she is 14 pixels up. Every joint is then mixed with that one dial:

```
hipF  = mixf(hipF,  48.0f,  air);    // 0 = running angle, 1 = leaping
angle
kneeF = mixf(kneeF, -55.0f, air);
armF  = mixf(armF,  60.0f,  air);
```

Halfway up she is literally half running and half leaping. That is why the change never looks like a jolt.

How the Desert Runner Works



The Scoreboard

The strip across the top is drawn by `drawHUD()`.

```
void drawHUD()
{
    glColor4f(0.10f, 0.16f, 0.20f, 0.30f);
    drawRect(0, 408, 800, 450);

    int total = (int)gElapsed;
    char buf[32];
    snprintf(buf, sizeof buf, "TIME  %02d:%02d", total / 60, total % 60);

    glColor3f(1.0f, 1.0f, 1.0f);
    drawText(20, 421, GLUT_BITMAP_HELVETICA_18, buf);

    drawText(650, 421, GLUT_BITMAP_HELVETICA_18, "LIVES");
    for (int i = 0; i < 3; i++)
        drawHeart(726 + i * 24, 428, 1.6f, i < gLives);
}
```

Line	What it does
<code>glColor4f(..., 0.30f)</code>	the fourth number is see through. 0.30 means mostly transparent
<code>snprintf(buf, ...)</code>	turns 92 seconds into the text <code>01:32</code>
<code>drawText(20, 421, ...)</code>	stamps letters using GLUT's built in font
<code>drawHeart(726 + i * 24, ...)</code>	three hearts, 24 apart. <code>i < gLives</code> decides full or empty

And a heart is just two circles and a triangle:

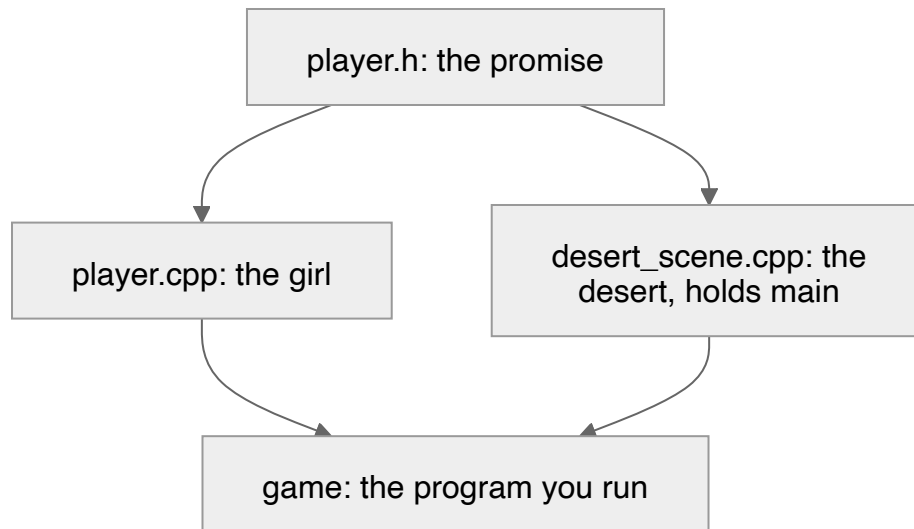
```
static void drawHeart(float cx, float cy, float s, int filled)
{
    if (filled) glColor3f(0.93f, 0.20f, 0.32f);
    else        glColor4f(0.25f, 0.16f, 0.16f, 0.45f);

    drawEllipse(cx - 3.2f * s, cy + 2.0f * s, 4.0f * s, 4.0f * s);
    drawEllipse(cx + 3.2f * s, cy + 2.0f * s, 4.0f * s, 4.0f * s);
    glBegin(GL_TRIANGLES);
        glVertex2f(cx - 6.8f * s, cy + 2.4f * s);
        glVertex2f(cx + 6.8f * s, cy + 2.4f * s);
        glVertex2f(cx,          cy - 7.0f * s);
    glEnd();
}
```

One honest note. **The hearts never go down yet.** Nothing checks whether the girl hits a tumbleweed, so she can run through them all day. The tool to fix this already exists: `playerHeight()` reports how high she is, so a future version can ask “was she still on the ground when the tumbleweed reached her?” and take a heart away.

Two Files, One Game

The girl lives in her own file so she can be used in other scenes later. Three files work together.



How the three files fit together

The header is a menu

```
#ifndef PLAYER_H           // include guard: stops the file being pasted
                             in twice
#define PLAYER_H

// Advance the run cycle and the jump physics.
// Call once per frame, dt = seconds since the last frame (0.016f at 60
// fps).
void playerUpdate(float dt);

// Start a jump. Ignored if she is already in the air.
// Wire this to the space bar in your keyboard() callback.
void playerJump();

// Draw her standing on ground level y, facing right.
// The jump height is added internally, so pass the ground y, not her
// feet.
void drawPlayer(float x, float y);

// How far off the ground she currently is, in pixels (0 while running).
// Useful for collision checks -- e.g. did she clear that tumbleweed?
float playerHeight();

// Run speed, in radians of cycle per second (9.0 is a steady jog).
void playerSetSpeed(float radiansPerSecond);

#endif
```

`player.h` is like a menu in a restaurant. The menu lists what you can order. It contains no food at all. The kitchen, `player.cpp`, is where the cooking happens.

This is why writing `#include "player.h"` on its own shows nothing. You read the menu but never ordered. You have to **call** the function, and you have to hand the compiler **both** files:

```
g++ desert_scene.cpp player.cpp -o game
```

Leave `player.cpp` off that line and the compiler says `Undefined symbols`, which means “you promised me this function existed and I cannot find it anywhere”.

The word `static` means private

Both files had a function called `drawRect`. Two functions with the same name made the build fail with `duplicate symbol`.

Putting `static` in front of the one inside `player.cpp` fixed it.

```
static void drawRect(float x1, float y1, float x2, float y2)
```

`static` means “this belongs to my file only, nobody else may use it”. Like writing your name on your lunchbox.

Every Function, One Line Each

desert_scene.cpp

Function	What it does
<code>wrapX</code>	the conveyor belt. Slides a thing left, wraps it back to the right
<code>drawRect</code>	a box
<code>drawEllipse</code>	an oval made of 28 tiny straight lines
<code>drawCapsule</code>	a box with a round top, used for cactus arms
<code>drawSky</code>	the blue green wash, darker at the top
<code>drawSun</code>	three circles inside each other, faint to bright, so it glows
<code>drawWave</code>	the pale wavy haze lines drifting across the sky
<code>drawCloud</code>	a fluffy cloud, dark underneath and white on top so it looks solid
<code>drawWispCloud</code>	thin see through streaks higher up
<code>drawBird</code>	a small V shape whose wings flap using <code>sin</code>
<code>drawHill</code>	one purple hill made of a wavy line filled in below
<code>drawGround</code>	brown dirt, wavy streaks, pebbles, and the pale sand band
<code>drawRock</code>	a rock, its shadow, and a light patch on top
<code>cactusArm</code>	one bent cactus arm: out, round the elbow, then up
<code>drawCactus</code>	the big cactus. Trunk, two arms, ribs, spines, flowers
<code>drawBarrelCactus</code>	the little round cactus with a flower
<code>drawSmallCactus</code>	the flat pad cactus with red fruit
<code>drawTumbleweed</code>	a tangled ball that spins as it travels and bounces
<code>drawSkull</code>	the cow skull lying in the dirt

Function	What it does
<code>drawText</code>	writes letters on screen
<code>drawHeart</code>	one heart, full or empty
<code>drawHUD</code>	the whole top strip
<code>display</code>	the artist. Draws one complete picture, in order
<code>tick</code>	the page turner. Runs 60 times a second
<code>keyboard</code>	listens for space, P, R and Escape
<code>reshape</code>	keeps the picture correct if you resize the window
<code>init</code>	sets up the graph paper and the colours, once at the start
<code>main</code>	opens the window and starts the loop

player.cpp

Function	What it does
<code>mixf</code>	the mixing dial. Blends between two numbers
<code>drawRect</code>	a box, private to this file
<code>drawCircle</code>	a circle, private to this file
<code>drawShoe</code>	one shoe
<code>drawLeg</code>	thigh, knee, shin, shoe, all turnable
<code>drawArm</code>	upper arm, elbow, forearm, hand
<code>drawBodice</code>	the top half of the dress
<code>drawDress</code>	the skirt, lifting and flowing
<code>drawDust</code>	sand puffs behind her feet
<code>drawLandPuff</code>	the landing puff
<code>drawPlayerPose</code>	draws her whole body in one exact pose
<code>playerUpdate</code>	moves time forward. Legs, jump, gravity, landing
<code>playerJump</code>	starts a jump, if she is on the ground
<code>playerHeight</code>	how high she is, for future collision checks
<code>playerSetSpeed</code>	how fast she runs
<code>drawPlayer</code>	the front door. Works out the pose and draws her

Six more functions (`display` , `tick` , `keyboard` , `reshape` , `init` , `main`) exist only when you build her preview on her own. They are wrapped in `#ifdef PLAYER_DEMO` so they cannot clash with the desert's versions.

Build It and Run It

The game:

```
g++ desert_scene.cpp player.cpp -o game -framework OpenGL -framework GLUT
./game
```

The girl on her own, with no desert:

```
g++ -DPLAYER_DEMO player.cpp -o player -framework OpenGL -framework GLUT
./player
```

Key	Does
Space	jump
P	pause and unpause
R	reset the clock and the hearts
Escape	quit

Try It Yourself

The best way to understand a program is to break it on purpose. Change one number, build it again, and watch. Every one of these is safe.

Change this	In	You will see
<code>JUMP_V</code> from 330 to 600	player.cpp	she jumps far too high
<code>GRAVITY</code> from 900 to 200	player.cpp	she floats down like she is on the moon
<code>gRate</code> from 9 to 20	player.cpp	her legs go wild
<code>32 * sin(p)</code> to <code>70 * sin(p)</code>	player.cpp	enormous cartoon strides
<code>hem = 22</code> to <code>hem = 8</code>	player.cpp	her long dress comes back and hides her legs
<code>28</code> to <code>5</code> in <code>drawEllipse</code>	desert_scene.cpp	every circle turns into a pentagon
speed 95 to 400 in <code>drawGround</code>	desert_scene.cpp	the ground rushes past under her feet
move <code>drawHUD()</code> above <code>drawSky()</code>	desert_scene.cpp	the scoreboard vanishes behind the sky
delete <code>glPopMatrix()</code> in <code>drawLeg</code>	player.cpp	her whole body tips over, because nobody went back

The last two are the best experiments in the list. One shows you what “the last one wins” really means. The other shows you exactly what `glPushMatrix` and `glPopMatrix` were quietly doing the whole time.